

Uncertainty-Conditioned Reinforcement Learning with Evidential Policy Networks for Autonomous Systems

Antonio Galdes
University of Malta
Faculty of ICT
antonio.galdes.22@um.edu.mt

ABSTRACT

Autonomous parking systems trained with reinforcement learning typically assume reliable localisation, yet sensor quality fluctuates unpredictably in practice. This report presents an uncertainty-conditioned reinforcement learning architecture that addresses this limitation through two complementary mechanisms. The first embeds six Extended Kalman Filter (EKF) posterior covariance elements (the three marginal standard deviations and three cross-covariance terms) directly into the policy observation vector. The second replaces the standard Gaussian actor head with a Normal-Inverse-Gamma (NIG) evidential output layer, yielding per-action epistemic and aleatoric uncertainty in a single forward pass. Together, these methods create an uncertainty-conditioned policy that is trained via Proximal Policy Optimisation (PPO) in the CARLA simulator under dynamically changing fix-states of the Real-Time Kinematic Global Navigation Satellite System (RTK-GNSS), which are represented by a discrete-time Markov chain. Moreover, a 2×2 factorial ablation study is proposed to isolate the contribution of each uncertainty layer to collision avoidance under degraded localisation.

Keywords

Reinforcement learning, evidential deep learning, uncertainty quantification, autonomous parking, localisation uncertainty, Extended Kalman Filter

1. INTRODUCTION

In the autonomous-driving sector, reinforcement learning (RL) is frequently utilised for vehicle control [1], usually on the assumption that the localisation system delivers accurate pose estimates throughout operation. In practice, RTK-GNSS accuracy deteriorates from centimetre to metre level depending on satellite visibility and atmospheric conditions. Additionally, automotive functional safety standards like ISO 26262 [2] prohibit the deployment of a parking system that collides with obstacles when localisation quality deteriorates. As a result, policies that were trained without considering the quality of localisation are unable to change when circumstances require it.

This problem is influenced by two different kinds of uncertainty. According to Kendall and Gal [3], aleatoric uncertainty captures irreducible noise in the environment and

persists regardless of data quantity [4], whereas epistemic uncertainty reflects the model’s ignorance of unfamiliar states and can be reduced with additional training data. Both these types of uncertainty can be quantified through evidential deep learning (EDL), a framework that has been applied to the value-estimation component of actor-critic architectures [5], to perception modules feeding model predictive controllers that optimise a finite-horizon action sequence at each step [6] and to value-based discrete-action agents [7].

Conditioning policies on uncertainty has also shown promise outside the EDL framework: uncertainty has been shown to provide a reliable signal for guiding agents in active exploration [8], and localisation uncertainty alone can guide policy behaviour without evidential outputs [9]. Nevertheless, neither the actor of a continuous-action policy-gradient agent nor the raw Extended Kalman Filter (EKF) covariance elements have been provided as RL observation dimensions for vehicle control. The central question is whether incorporating localisation uncertainty, through pose covariance observation, evidential policy outputs, or both, yields measurably safer behaviour under degraded conditions.

The proposed system addresses this gap by placing an evidential NIG head on a continuous-action policy-gradient actor and feeding the EKF posterior covariance as an observation input. It will be evaluated on an outdoor parking task in the CARLA driving simulator [10] under dynamically changing GNSS quality.

2. AIMS AND OBJECTIVES

This study aims to create and assess an uncertainty-conditioned RL architecture that enables an autonomous parking agent to degrade gracefully under declining localisation quality, with the following objectives:

1. Implement an EKF-based localisation pipeline within CARLA that fuses RTK-GNSS and Inertial Measurement Unit (IMU) data via the `robot_localization` ROS 2 package [11] and produce a posterior covariance matrix reflecting sensor degradation. Correct operation will be assessed by verifying that steady-state covariance magnitude, averaged over repeated trials, increases monotonically in expectation across the four RTK fix-state tiers.
2. Develop an evidential Proximal Policy Optimisation (PPO) algorithm [12] in which the actor network outputs Normal-Inverse-Gamma (NIG) distributions over

continuous actions [13], enabling single-pass quantification of epistemic and aleatoric uncertainty. The decomposition will be assessed by comparing uncertainty outputs across in-distribution and out-of-distribution conditions.

3. Conduct a 2×2 ablation study comparing the full method against baselines lacking one or both uncertainty layers, measuring success rate and collision rate under both nominal and degraded GNSS conditions to isolate the contribution of each uncertainty layer.

3. BACKGROUND AND LITERATURE REVIEW

3.1 Reinforcement Learning for Vehicle Control

The Markov Decision Process (MDP) for sequential decision-making is defined by a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, P the transition function, R the reward function and $\gamma \in [0, 1)$ the discount factor [14, 15]. At each time step t , the agent observes state $s_t \in \mathcal{S}$, selects action $a_t \in \mathcal{A}$, receives scalar reward r_t and transitions to successor state s_{t+1} . A policy $\pi_\theta(a|s)$ then maps states to action distributions, parameterised by neural network weights θ , and the objective is to determine θ such that the expected discounted return $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$ is maximised.

Policy gradient approaches use gradient ascent to optimise θ on this objective, but uncontrolled updates might result in unfavourable policy changes. This is reduced by PPO [12], which uses a clipped surrogate target to limit the scope of policy updates:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right] \quad (1)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ denotes the probability ratio and $\hat{A}_t$ is the advantage estimate. This estimate is obtained via Generalised Advantage Estimation (GAE), which interpolates between single-step temporal-difference bootstraps and full-episode Monte Carlo return estimates using a trace-decay parameter λ [16].

The complete training objective has three terms. The clipped surrogate from Eq. (1) optimises the actor, an entropy bonus [17] discourages premature convergence to a deterministic policy, and a value function loss [12] trains the critic to produce advantage estimates on which the actor depends. This loss is defined as the squared residual between the critic’s state-value estimate $V_\theta(s_t)$ and the empirical return $\hat{R}_t$:

$$L^{\text{VF}}(\theta) = \left(V_\theta(s_t) - \hat{R}_t \right)^2. \quad (2)$$

Minimising this loss regresses the critic toward observed returns, providing the advantage estimates that drive the clipped policy objective.

PPO improves data efficiency over single-pass policy gradient approaches by gathering transitions across a fixed-length rollout, dividing them into mini-batches, and executing several gradient update epochs per rollout [12]. In practice, training stability is further reinforced by two complementary measures: monitoring approximate KL divergence as a diagnostic early-stopping signal, and normalising observations

via running statistics when input dimensions span different scales [18]. These on-policy rollout properties make PPO a natural fit for the proposed system, as the clipped surrogate keeps policy stability and uncertainty quantification as separate concerns.

3.2 Evidential Deep Learning

Instead of learning point estimates, evidential deep learning sets a prior over the likelihood parameters. For regression, the target y is modelled as being drawn from a Gaussian likelihood $\mathcal{N}(\mu, \sigma^2)$, and an NIG prior is imposed jointly over the unknown mean μ and variance σ^2 [13]. For each output dimension, the network produces four parameters $(\gamma, \nu, \alpha, \beta)$, where γ is the predicted mean of μ , $\nu > 0$ is the pseudo-observation count controlling the posterior concentration over μ , and $\alpha > 1$, $\beta > 0$ parameterise the inverse-gamma prior over the observation variance σ^2 . Two uncertainty types are then obtained in closed form:

$$\sigma_{\text{alea}}^2 = \frac{\beta}{\alpha - 1}, \quad \sigma_{\text{epi}}^2 = \frac{\beta}{\nu(\alpha - 1)}. \quad (3)$$

The aleatoric term is irreducible, whilst the epistemic term quantifies model uncertainty over γ and shrinks as ν grows with evidence [3].

When evidential deep learning was first developed for classification [19], it used a Dirichlet prior over class probabilities instead of generating point estimates. The framework was later extended to continuous regression through the NIG formulation [13], with the resulting uncertainty decomposition given in Eq. (3). Since then, a gradient suppression effect has been discovered [20]. As epistemic uncertainty grows, the gradient of the supervised evidential loss with respect to the predictive mean diminishes, causing the network to express uncertainty rather than refine its predictions. This problem is made worse in reinforcement learning, when the supervised penalty term is ill-defined due to the lack of ground-truth action targets. To address this, the proposed formulation is to add a prior-anchoring regularisation, which substitutes a limited log-penalty anchored to the NIG hyperprior for the supervised penalty.

Within reinforcement learning, evidential methods have so far been applied to the critic rather than the actor. Akgül et al. [5] address the non-stationarity of the target value distribution by placing an NIG evidential prior on the value function $V(s)$, whilst the actor retains a standard Gaussian policy. Stutts et al. [7] extend this to discrete-action contexts, pairing evidential Q-value estimation with quantile calibration to produce sample-free global uncertainty estimates. Beyond policy-gradient settings, a distinct line of work couples evidential uncertainty with an optimisation-based controller rather than a learnt policy [6], applying NIG-parameterised evidential deep learning under distributionally robust Model Predictive Control (MPC) constraints within CARLA.

In contrast to these single-pass evidential approaches, ensemble-based alternatives [21] require multiple forward passes to obtain uncertainty estimates. In the MuJoCo simulator [22], robustness in novel scenarios can be improved by conditioning the policy on ensemble disagreement [23]. In the autonomous driving domain, the Ensemble Quantile Networks method [24] combines per-quantile value distributions with independently initialised ensemble members to separately estimate aleatoric and epistemic uncertainty. The

epistemic signal is then used to withhold decisions when out-of-distribution situations are encountered, eliminating most collisions outside the training distribution. Monte Carlo Dropout [25] reduces this to a single model but still requires several stochastic passes at inference time. The proposed single NIG forward pass avoids all of these overheads, generating both types of uncertainty concurrently and making it well-suited to control policies operating at high frequencies with low inference latency.

3.3 Extended Kalman Filter Localisation

The Kalman Filter (KF) is an optimal recursive Bayesian estimator for linear systems subject to Gaussian noise, propagating a Gaussian belief over the state through alternating prediction and update steps [15]. Vehicle motion models however, are inherently nonlinear making the KF inapplicable. The EKF overcomes this by linearising the motion and measurement models via first-order Taylor expansion, substituting system matrices with their Jacobians at the current state estimate.

The EKF provides a principled means of fusing heterogeneous sensors such as GNSS and IMU into a continuous, covariance-bounded pose estimate, maintaining a Gaussian belief over the vehicle state $\mathbf{x}_t = [x, y, \psi]^\top$, where x and y are the planar position coordinates and ψ is the heading angle (the full `robot_localization` state also includes linear and angular velocities, but the planar pose subspace suffices to define the 3×3 covariance structure used here). The prediction step inflates the covariance according to process noise $\mathbf{Q}$:

$$\mathbf{P}_{t|t-1} = \mathbf{F}_t \mathbf{P}_{t-1|t-1} \mathbf{F}_t^\top + \mathbf{Q}, \quad (4)$$

where $\mathbf{F}_t$ is the state transition Jacobian, $\mathbf{P}_{t-1|t-1}$ is the posterior covariance from the previous step and $\mathbf{Q}$ is the process noise covariance. The update step then incorporates a measurement with noise covariance $\mathbf{R}$:

$$\mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \mathbf{P}_{t|t-1}, \quad (5)$$

where $\mathbf{K}_t$ is the Kalman gain, $\mathbf{H}_t$ is the measurement Jacobian and $\mathbf{I}$ is the identity matrix. The covariance $\mathbf{P}_{t|t}$ therefore grows between GNSS fixes and collapses upon each arriving measurement, producing a characteristic sawtooth profile. This time-varying posterior covariance is published alongside the pose estimate at each filter step by the `robot_localization` package [11], originally described for ROS 1 and subsequently ported to ROS 2, making it accessible to downstream components that condition on localisation uncertainty. The informativeness of this covariance signal is sensitive to process noise configuration, however, as Kovacs et al. [26] demonstrated, misconfigured process noise significantly degrades EKF localisation accuracy. Therefore, establishing careful noise parameterisation is a prerequisite for reliable covariance estimation.

3.4 Research Gap

Having surveyed the three foundations of the proposed system, the remaining gap can now be made explicit. On the input side, in Puthumanai et al. [9], task-specific uncertainty maps (TSUMs) are created from localisation quality and fed to a Soft Actor-Critic (SAC) agent [27], an off-policy entropy-regularised algorithm. This TSUM abstraction eliminates the geometric complexity of the complete posterior

covariance by compressing localisation uncertainty into a scalar or low-dimensional map. In contrast, exposing the raw EKF covariance elements directly preserves the off-diagonal terms P_{xy} , $P_{x\psi}$ and $P_{y\psi}$, which encode directional correlations between position and heading uncertainty, and avoids constructing a separate map.

On the output side, as argued in Charpentier et al. [4], epistemic and aleatoric uncertainty can be meaningfully disentangled in the RL setting through appropriate architectural choices, providing motivation for using the NIG decomposition to distinguish model ignorance from irreducible action noise at deployment. A gap is therefore identified: no evidential NIG head has ever been placed on the actor of a continuous-action policy-gradient agent, while the EKF posterior covariance, including off-diagonal cross-covariance terms, has never been exposed as explicit observation dimensions for ego-vehicle control.

4. METHODOLOGY

4.1 Common Infrastructure

The system is deployed as three Docker containers orchestrated via Docker Compose, illustrated in Fig. 1. The physics engine and sensor models are provided by the `carla-server` container, which runs CARLA 0.9.16 [10] headlessly with NVIDIA GPU passthrough. The `ros2-bridge` container runs ROS 2 Jazzy, hosting both the CARLA-ROS bridge and the `robot_localization` EKF node [11], into which a custom node injects tier-dependent covariance by intercepting GNSS messages prior to fusion. Lastly, the training container runs PyTorch with Stable-Baselines3 [18], subscribing to EKF output via Data Distribution Service (DDS) with a Gymnasium-compatible environment wrapper.

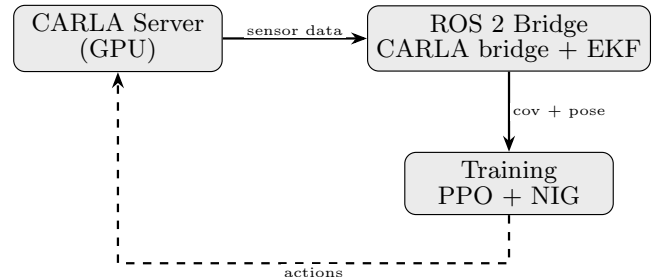


Figure 1: Three-container Docker architecture.

Layout modules, each of which specifies a floor plan shape in a local coordinate frame, define the geometry of parking lots. Three shapes have been implemented: a 60×45 m axis-aligned rectangle, a trapezium (front width 48 m, rear width 30 m, length 44 m), and a nine-sided irregular polygon. The rectangle and trapezium will be used for training while the irregular polygon is reserved for out-of-distribution (OOD) evaluation to assess whether epistemic uncertainty increases. Further layouts can be added via the same interface.

Each module specifies spawn locations, patrol waypoints, pedestrian zone boundaries, perimeter cone positions interpolated along the polygon edges, and bay positions (perpendicular, angled, and parallel types according to the dimensional standards of the German parking design guideline EAR 05 [28]). An orchestrator script then applies a world-frame transform, producing bird’s-eye PNG plots for visual

verification (Fig. 2) as well as YAML layout files used by the training environment.

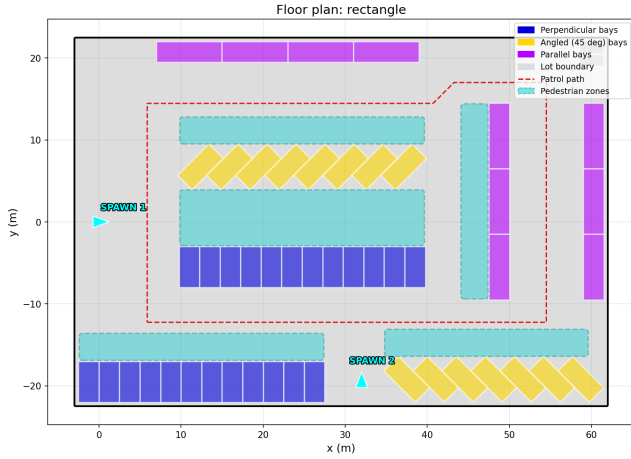


Figure 2: Generated bird's-eye view of the rectangle floor plan showing bay positions (colour-coded by type), spawn points, patrol path and pedestrian zones.

A unified inspector script provides four options for evaluating the system prior to training, all of which run inside dedicated Docker containers. In layout mode, the entire parking lot is created in windowed CARLA with geometry overlays presented as persistent debug primitives: spawn points, pedestrian zone boundaries, the patrol waypoint path, target bay highlighted in green, and bay outlines colour-coded by type. In sensors mode, the ego vehicle is spawned with each sensor mount point specified by a coloured dot, as illustrated in Fig. 3, and the 2D LiDAR is also annotated with a field-of-view arc. In live mode, the obstacle detection range is visualised by rendering the 2D LiDAR scan returns as debug dots in the CARLA world. Lastly, dryrun mode allows the EKF localisation pipeline and observation construction to be tested end-to-end before any training is undertaken by running the whole training pipeline in windowed CARLA without a trained model, driven via teleoperation using a keyboard.



Figure 3: Sensor inspector side-profile view indicating IMU, GNSS antenna and 2D LiDAR mount positions on the vehicle body.

On the other hand, a 2D bird's-eye visualiser is also implemented to evaluate the system during training. This works by using Pygame to render lot boundaries, bay outlines, NPC

placements, and the ego vehicle trajectory trail by following a per-step log file written by the training environment (Fig. 4). To avoid additional overhead during training, a signal file is attached and detached to the visualiser at runtime, allowing evaluation to be performed without having to restart training. Furthermore, a 3D spectator mode offers a qualitative perspective on learnt driving behaviour by loading a saved checkpoint and replaying evaluation episodes in windowed CARLA.

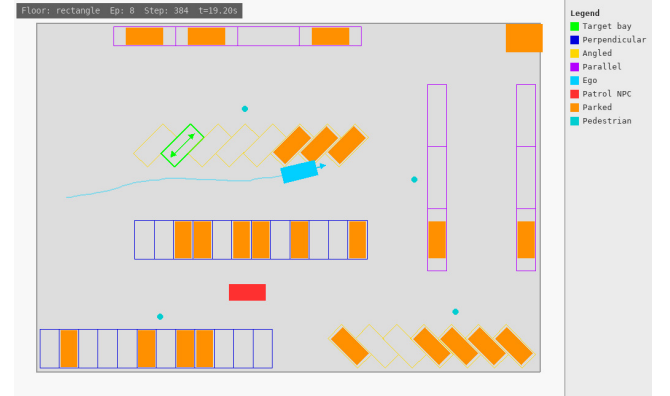


Figure 4: Live bird's-eye training visualiser showing ego trajectory, bay outlines and NPC vehicles.

The observation vector has 17 dimensions divided into four semantic groups, summarised in Table 1. EKF velocity supplies the motion state for deceleration and alignment, the EKF covariance group exposes posterior uncertainty so the policy can condition on localisation confidence, and the relative target $(dx, dy, d\psi)$ encodes navigational intent in the ego body frame.

Table 1: Observation space structure (17 dimensions).

Indices	Features	Source
0-2	$v_x, v_y, \dot{\psi}$	EKF velocity
3-5	$\sqrt{P_{xx}}, \sqrt{P_{yy}}, \sqrt{P_{\psi\psi}}$	EKF std. devs
6-8	$P_{xy}, P_{x\psi}, P_{y\psi}$	EKF cross-cov
9-11	$dx, dy, d\psi$	Relative target
12-16	left/right/fwd clearance	2D LiDAR

The fourth group, hemispheric LiDAR clearance, provides five features from a single-plane 2D LiDAR [29]. The left and right hemispheres each contribute a nearest-obstacle distance and bearing, and the forward arc contributes distance alone given that its narrow 30-degree cone renders bearing redundant. The sensor is mounted at front-bumper height (0.5m above ground) so that the scan plane resolves perimeter cones, parked vehicles and pedestrians at heights characteristic of a parking environment.

The six covariance features comprise the three marginal standard deviations $\sqrt{P_{ii}}$ and the three off-diagonal cross-covariance terms P_{xy} , $P_{x\psi}$ and $P_{y\psi}$. The standard deviations provide a direct, interpretable measure of position and heading uncertainty in physical units, whilst the cross-covariance terms encode directional correlations between position and heading uncertainty that cannot be recovered from the marginal statistics alone. A signed log1p transfor-

mation, $\text{sign}(x) \log(1 + |x|)$, is applied to all six features to compress heavy tails whilst preserving the sign of off-diagonal terms. CARLA ground truth is only employed for reward computation and never appears in the observation, ensuring the policy does not rely on simulator state.

The action space is two-dimensional and continuous. The first dimension is steering, $\delta \in [-1, 1]$, varying continuously across that range. The second dimension is a signed longitudinal command, $u \in [-1, 1]$, where positive values correspond to throttle and negative values to braking, with the environment decomposing this signal into separate CARLA throttle and brake actuator inputs at runtime.

The initial reward function employs distance-based shaping inspired by potential-based shaping [30] to guide the agent toward the target bay:

$$r_t = \frac{d_{t-1} - d_t}{d_{\max}} - c_{\text{step}}, \quad (6)$$

where d_t is the Euclidean distance to the target bay at step t , $d_{\max}$ is a fixed normalisation constant set to the approximate diagonal of the largest training lot and c_{step} is a small per-step penalty. The formulation adapts the standard potential-based framework [30] by replacing a state-dependent potential denominator with a fixed scale factor suited to the CARLA simulator. Beyond the dense per-step shaping signal, terminal rewards of +10 and -10 are awarded for success and collision respectively, where success requires position error < 0.5 m, orientation error $< 10^\circ$ and speed < 0.1 m/s simultaneously.

To avoid the policy memorising a single static configuration, dynamic actors are added to each training episode. A single NPC patrol vehicle follows waypoint loops through the lot aisles via a proportional heading controller, braking when another actor is detected within a configurable forward-cone stop distance.

Within each designated pedestrian zone, a single pedestrian executes random walks and may be spawned with a configurable probability per episode, with zone confinement and ego-avoidance logic ensuring that any collision is only attributable to the ego vehicle.

The target bay is always maintained empty and bay occupancy is randomised per episode. For perpendicular and angled target bays, neighbouring bays may be occupied, whereas for parallel bays they are kept vacant, as the agent lacks reverse functionality and forward-only entry requires lateral clearance. Finally, traffic cones are spaced consistently around the perimeter of the lot, and collisions with these result in episode termination with a collision penalty.

4.2 EKF-Based Localisation Pipeline

RTK-GNSS localisation quality is characterised as four discrete tiers, namely RTK fixed (~ 2 cm), RTK float (~ 30 cm), standalone (~ 2 m) and degraded (~ 5 m). Transitions between adjacent tiers are governed by a discrete-time Markov chain evaluated at the 20 Hz GNSS publishing rate, simulating gradual real-world variations in signal quality. These dynamics are encoded by the following transition probability matrix:

$$\mathbf{T} = \begin{bmatrix} 0.995 & 0.005 & 0 & 0 \\ 0.003 & 0.992 & 0.005 & 0 \\ 0 & 0.004 & 0.993 & 0.003 \\ 0 & 0 & 0.005 & 0.995 \end{bmatrix} \quad (7)$$

where T_{ij} denotes the probability of moving from tier i to tier j at the next time step, and transitions between non-adjacent tiers are forbidden. The high diagonal values ensure gradual rather than abrupt degradation, producing realistic mid-episode signal quality changes. These probabilities are deliberately chosen to over-represent degraded conditions during training, yielding a stationary distribution that places approximately 55% of simulation time in standalone or degraded modes; this ensures sufficient exposure to rare failure modes rather than serving as a literal model of RTK availability in unobstructed outdoor environments. Furthermore, realistic IMU process noise derived from the VectorNav VN-100 datasheet [31] is additionally configured in the EKF to ensure covariance inflates appropriately between GNSS fixes. Both noise sources will assess the policy's ability to condition on localisation uncertainty.

4.3 Evidential PPO Actor

The actor's final layer produces NIG parameters $(\gamma, \nu, \alpha, \beta)$ per action dimension, with γ serving as the predictive mean for action sampling, $a \sim \mathcal{N}(\gamma, \sigma_{\text{alea}}^2)$. Positivity is enforced through the softplus function, $\text{softplus}(x) = \log(1 + e^x)$, with offsets:

$$\begin{aligned} \nu &= \text{softplus}(z_\nu) + \epsilon \\ \alpha &= \text{softplus}(z_\alpha) + 1 \\ \beta &= \text{softplus}(z_\beta) + \epsilon \end{aligned} \quad (8)$$

where z_ν , z_α and z_β are the raw pre-activation outputs of the final network layer and $\epsilon = 10^{-6}$. The unit offset on α ensures finite variance in Eq. (3), with all three parameters additionally clamped to a maximum of 100 to prevent divergence in the RL setting, where no ground-truth action target exists. To prevent the NIG parameters from drifting arbitrarily far from their initialisations, a prior-anchoring log-penalty is applied:

$$\mathcal{L}_{\text{reg}} = \lambda \left[\overline{\log\left(\frac{\nu}{\nu_0} + 1\right)} + \overline{\log\left(\frac{\alpha}{\alpha_0} + 1\right)} \right] \quad (9)$$

where ν_0, α_0 are initialisation-time values, λ is linearly annealed from zero over a warmup period and $\overline{(\cdot)}$ denotes the mean over the batch and action dimensions. In contrast to the supervised regression regulariser of Amini et al. [13], this formulation is tailored for the RL situation, where $|a - \gamma|$ represents the intrinsic stochasticity of the policy instead of a prediction error against a known goal. These elements come together to form a single minimisation aim, and since L^{CLIP} was specified as a quantity to be maximised, adding it to the total loss negates it:

$$L = -L^{\text{CLIP}} + c_v L^{\text{VF}} - c_H H(\pi_\theta) + \mathcal{L}_{\text{reg}}, \quad (10)$$

where L^{CLIP} is the clipped surrogate objective from Eq. (1), L^{VF} is the value function loss from Eq. (2), c_v is the value function coefficient, $H(\pi_\theta)$ is the policy entropy, c_H is its coefficient and $\mathcal{L}_{\text{reg}}$ is the evidential regularisation term from Eq. (9).

The training loop implementing the total loss from Eq. (10) is presented in Algorithm 1. The NIG parameters $(\gamma, \nu, \alpha, \beta)$ are produced by the actor's forward pass during action evaluation within each mini-batch step and are then cached so

that $\mathcal{L}_{\text{reg}}$ can be computed without a redundant forward pass.

Algorithm 1 Evidential PPO training loop.

```

1: Load configuration from YAML
2: Initialise CARLA environment with EKF pipeline
3: Normalise observations via running statistics
4: Initialise EvidentialPPO with NIG actor head
5: for each rollout of  $n$  environment steps do
6:   Collect transitions  $(s_t, a_t, r_t, s_{t+1})$ 
7:   Compute advantages via GAE
8:   for each optimisation epoch do
9:     Sample mini-batch
10:    Evaluate actions and cache NIG parameters
11:    Compute  $L^{\text{CLIP}}$ ,  $L^{\text{VF}}$ ,  $H(\pi_\theta)$  and  $\mathcal{L}_{\text{reg}}$ 
12:    if approximate KL between  $\pi_{\theta_{\text{old}}}$  and  $\pi_\theta$  over the
        rollout exceeds  $\text{KL}_{\text{target}}$  then
13:      Early-stop epoch loop
14:    end if
15:    Backpropagate  $L$  and update  $\theta$ 
16:  end for
17:  Log uncertainty metrics to TensorBoard
18: end for
19: Save final model and normalisation statistics

```

Prior to the ablation study, Optuna is used to determine all hyperparameter values using a multivariate Tree-structured Parzen Estimator (TPE) sampler that models parameter interactions by estimating the joint density of high-performing configurations rather than treating each hyperparameter independently [32]. The search takes into account eight parameters: the learning rate, rollout length, mini-batch size, number of gradient epochs, entropy coefficient, evidential regularisation coefficient λ , discount factor γ , and its linear warmup time. The clipping range, gradient norm limit, KL target, and network architecture are held fixed, as large-scale sensitivity studies have identified these as low-variance dimensions across continuous control tasks [33].

The study will run 35 trials, each training for 100,000 steps and evaluated on mean episodic progress reward, which provides a dense signal even when the agent infrequently achieves a successful park. Unpromising trials are terminated early by a MedianPruner [34]. The best configuration is subsequently written to the training configuration file and applied without modification across all four ablation baselines.

4.4 2x2 Factorial Ablation Study

To separate the contribution of each uncertainty layer, a 2×2 factorial ablation study will be carried out, as detailed in Table 2. All four configurations will share identical PPO hyperparameters and each will be trained across three random seeds to assess seed-to-seed variability.

Table 2: Ablation study design.

	No EKF cov. (11d)	EKF cov. (17d)
Standard actor	vanilla_ppo	input_unc
Evidential actor	output_unc	full_method

5. EVALUATION PLAN

A deterministic policy is evaluated over 100 episodes per condition, with GNSS and IMU noise multipliers, dynamic actor density, and bay occupancy held fixed.

5.1 Localisation Pipeline Verification

In order to confirm that covariance inflates correctly with signal deterioration, root mean square error is calculated for position and heading and EKF pose predictions are compared to CARLA ground truth under each GNSS tier.

5.2 Uncertainty Behaviour

- Epistemic uncertainty on unseen layouts.** Mean per-step epistemic uncertainty on a held-out floor plan will be compared against the same configurations on a training layout under identical GNSS conditions, to assess whether the evidential head produces different uncertainty levels on novel geometry.
- Aleatoric uncertainty across GNSS tiers.** Mean per-step aleatoric uncertainty will be recorded at each GNSS tier to investigate whether the policy learns to express higher action-level aleatoric noise when localisation quality degrades.

5.3 Ablation Study Outcomes

The following metrics are planned, each averaged across seeds and reported with standard deviation:

- Success rate per GNSS tier.** A parking attempt will be defined as successful when the agent comes to rest within 0.5 m of the target bay centre, within 10° of the target heading and below 0.1 m/s, with all three criteria met simultaneously. Success rate will be computed as the fraction of episodes satisfying these criteria, reported per ablation cell across the four GNSS tiers (RTK fixed, RTK float, standalone and degraded).
- Collision rate under degraded GNSS.** The fraction of episodes in which a collision is registered by the CARLA physics engine under standalone (~ 2 m) and degraded (~ 5 m) noise conditions. This will be recorded for each ablation cell and compared across the 2×2 grid, following offline evaluation practices for driving models [35].
- Graceful degradation curve.** Success rate will be plotted against GNSS noise multiplier for each ablation cell, producing a degradation profile for comparison across configurations.

6. CONCLUSION

The core system infrastructure has been completed: the RTK-GNSS localisation pipeline, CARLA simulation environment, GNSS Markov noise model, evidential PPO agent, and supporting inspection and visualisation tooling are all in place. Each uncertainty layer is independently exercisable through the ablation design, and the controlled GNSS degradation conditions are established. The end-to-end pipeline is undergoing final integration testing, after which hyperparameter optimisation will commence, followed by the full ablation study.

7. REFERENCES

- [1] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda. A survey of autonomous driving: Common practices and emerging technologies. *IEEE Access*, 8:58443–58469, 2020.
- [2] International Organization for Standardization. ISO 26262:2018 road vehicles – functional safety. <https://www.iso.org/publication/PUB200262.html>, 2018.
- [3] A. Kendall and Y. Gal. What uncertainties do we need in Bayesian deep learning for computer vision? *arXiv preprint arXiv:1703.04977*, 2017. Also appeared at NeurIPS 2017.
- [4] B. Charpentier, R. Senanayake, M. Kochenderfer, and S. Günnemann. Disentangling epistemic and aleatoric uncertainty in reinforcement learning. *arXiv preprint arXiv:2206.01558*, 2022.
- [5] A. Akgül, G. Baykal, M. Haußmann, and M. Kandemir. Overcoming non-stationary dynamics with evidential proximal policy optimization. *arXiv preprint arXiv:2503.01468*, 2025. Accepted to TMLR 2025.
- [6] H. Ham and H. Ahn. DRO-EDL-MPC: Evidential deep learning-based distributionally robust model predictive control for safe autonomous driving. *arXiv preprint arXiv:2507.05710*, 2025.
- [7] A. C. Stutts, D. Erricolo, T. Tulabandhula, and A. R. Trivedi. Echoes of Socratic doubt: Embracing uncertainty in calibrated evidential reinforcement learning. *arXiv preprint arXiv:2402.07107*, 2024.
- [8] J. A. Placed and J. A. Castellanos. A deep reinforcement learning approach for active SLAM. *Appl. Sci.*, 10(23):8386, 2020.
- [9] G. Puthumanai, P. Padrao, J. Fuentes, L. Bobadilla, and M. Ornik. GUIDed agents: Enhancing navigation policies through task-specific uncertainty abstraction in localization-limited environments. *arXiv preprint arXiv:2410.15178*, 2025.
- [10] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun. CARLA: An open urban driving simulator. *arXiv preprint arXiv:1711.03938*, 2017. Also appeared at CoRL 2017.
- [11] T. Moore and D. Stouch. A generalized extended Kalman filter implementation for the Robot Operating System. In *Intell. Auton. Syst. 13 (IAS-13)*, pages 335–348, Cham, Switzerland, 2016. Springer.
- [12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [13] A. Amini, W. Schwarting, A. Soleimany, and D. Rus. Deep evidential regression. *arXiv preprint arXiv:1910.02600*, 2020. Also appeared at NeurIPS 2020.
- [14] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, USA, 2nd edition, 2018.
- [15] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, Cambridge, MA, USA, 2005.
- [16] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2016. Also appeared at ICLR 2016.
- [17] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. *arXiv preprint arXiv:1602.01783*, 2016. Also appeared at ICML 2016.
- [18] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *J. Mach. Learn. Res.*, 22(268):1–8, 2021.
- [19] M. Sensoy, L. Kaplan, and M. Kandemir. Evidential deep learning to quantify classification uncertainty. *arXiv preprint arXiv:1806.01768*, 2018. Also appeared at NeurIPS 2018.
- [20] D. Oh and B. Shin. Improving evidential deep learning via multi-task learning. *arXiv preprint arXiv:2112.09368*, 2021. Also appeared at AAAI 2022.
- [21] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *arXiv preprint arXiv:1612.01474*, 2017. Also appeared at NeurIPS 2017.
- [22] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pages 5026–5033, 2012.
- [23] C. Kim, J.-K. Cho, H.-S. Yoon, S.-W. Seo, and S.-W. Kim. UNICON: Uncertainty-conditioned policy for robust behavior in unfamiliar scenarios. *IEEE Robot. Autom. Lett.*, 7(4):9099–9106, 2022.
- [24] C.-J. Hoel, K. Wolff, and L. Laine. Ensemble quantile networks: Uncertainty-aware reinforcement learning with applications in autonomous driving. *IEEE Trans. Intell. Transp. Syst.*, 24(6):6030–6041, 2023.
- [25] Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. *arXiv preprint arXiv:1506.02142*, 2016. Also appeared at ICML 2016.
- [26] I. Kovacs, M. Stefanoni, R. Pesti, D. Csik, P. Sarcevic, and A. Odry. Evaluation of optimal covariance models for EKF-based wheeled mobile robot localization. *Acta Polytechnica Hungarica*, 22(12):143–158, 2025.
- [27] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018. Also appeared at ICML 2018.
- [28] Forschungsgesellschaft für Straßen- und Verkehrswesen. EAR 05: Empfehlungen für anlagen des ruhenden verkehrs [recommendations for stationary traffic facilities], 2005. FGSV 283.
- [29] SICK AG. *TiM571-2050101 2D LiDAR Sensors: Data Sheet*. SICK AG, Waldkirch, Germany, 2026. Order no. 1075091.
- [30] A. Y. Ng, D. Harada, and S. J. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proc. 16th Int. Conf. Mach. Learn. (ICML)*, pages 278–287. Morgan Kaufmann, 1999.
- [31] VectorNav Technologies. *VN-100 Rugged IMU/AHRS: Sensor Datasheet (Hardware v7.0)*. VectorNav Technologies, LLC, Dallas, TX, USA, 2023. Doc. DS100-CR-70, Version 25-100-CR-70-R1.
- [32] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.
- [33] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem. What matters for on-policy deep actor-critic methods? A large-scale study. In *Proc. 9th Int. Conf. Learning Representations (ICLR)*, 2021.
- [34] T. Eimer, M. Lindauer, and R. Raileanu. Hyperparameters in reinforcement learning and how to tune them. In *Proc. 40th Int. Conf. Machine Learning (ICML)*, pages 9104–9149, 2023.
- [35] F. Codevilla, A. M. López, V. Koltun, and A. Dosovitskiy. On offline evaluation of vision-based driving models. *arXiv preprint arXiv:1809.04843*, 2018. Also appeared at ECCV 2018.

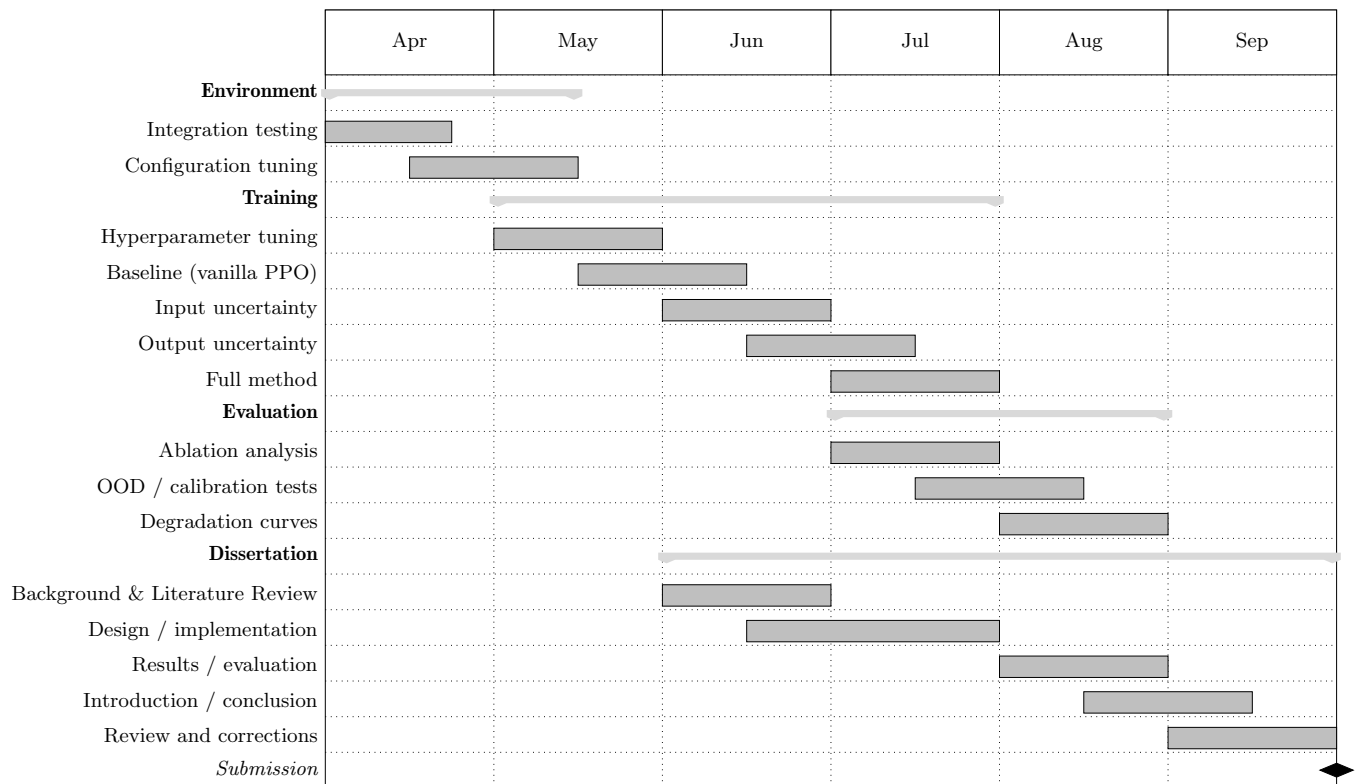


Figure 5: Dissertation work plan (April-September 2026).